Chapter 6: Modular Programming-Cont

Mohamed M. Sabry Aly
N4-02c-92

1

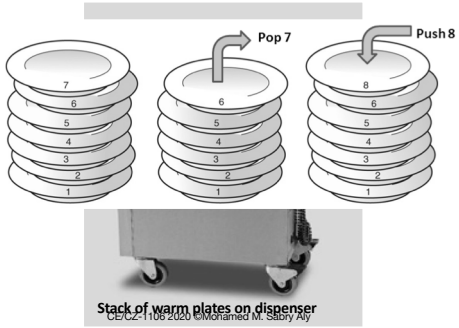
Learning Objectives

- List the stack manipulation operations & its implementation
- Describe passing parameters to subroutines using the stack
- Identify the difference between passing by value and by reference.
- Describe how a transparent subroutine can be implemented.

2

System Stack

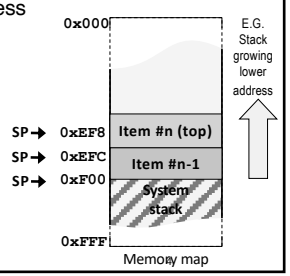
- A stack is a first-in, last-out linear data structure that is maintained in the memory's data area.



3

System Stack

- A stack is a first-in, last-out linear data structure that is maintained in the memory's data area.
  - The system stack in the ARM processor is maintained by a dedicated stack pointer (**SP, R13**).
  - Stack can grow towards lower or higher memory address
    - Preferred direction is lower, start from max address → Why?
  - The **SP** points to the top item on the system stack.
  - The 3 basic stack operations are **push**, **pop** and **access** items on the stack.

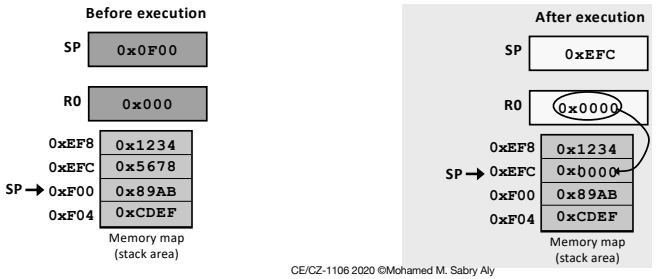


Ref: Null & Lobur (3rd Ed) section A.2.3 – Stacks

4

Push Data to the Stack

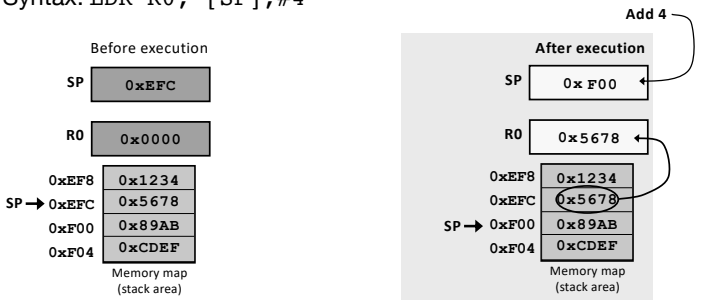
- Writing to stack can be done using **`STR`** instruction
  - Need to also increase the stack pointer before storing
- Syntax: `STR R0, [SP,#-4]!`



5

Pop Data off the Stack

- Popping from the stack can be done with **`LDR`** instruction
  - Need to update pointer AFTER read
- Syntax: `LDR R0, [SP],#4`

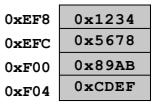


6

Removing from stack

- After popping, is the data **erased** from the stack?

NO



- Data still resides in memory and can be read

`LDR R0, [SP, #-4]` SP is not changed

7

Pushing Registers to Stack

- E.g., want to push R0 and R1

Method 1:

```
STR R0, [SP, #-4]!
STR R1, [SP, #-4]!
```

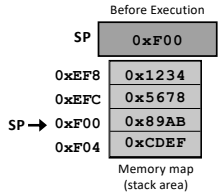

8

Pushing Registers to Stack

- E.g., want to push R0 and R1

Method 1:

```
STR R0, [SP, #-4]!
STR R1, [SP, #-4]!
```



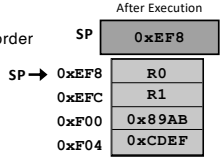
Method 2:

Use Stack pointer and write back updated value after operation

STMFD SP!,{R1,R0}

Write registers in descending order

Store multiple registers fully descending



9

Pushing and popping to stack

```
STMFD SP!, {list of registers}

LDMFD SP!, {list of registers}
```

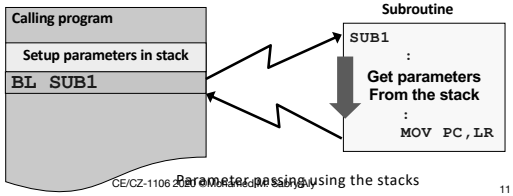
STMFD SP!,{R1,R0}

LDMFD SP!,{R1,R0}

10

Parameter Passing using Stack

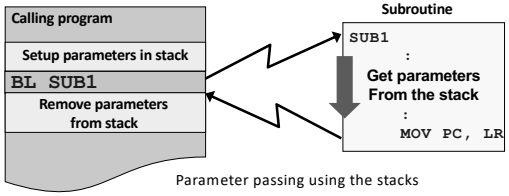
- Parameters are pushed onto the stack before calling the subroutine and retrieved from the stack within the subroutine.
- Most **general** method of parameter passing – no registers needed, supports recursive programming.
- **Large** numbers of parameters can be passed as long as stack does not overflow.



Parameter passing using the stacks

11

Parameter Passing using Stack



Parameter passing using the stacks

- Parameters pushed to the stack must be **removed** by the calling program immediately after returning from subroutine.
- If not, repeated pushing of parameters to the stack will lead to a stack overflow.

12

Sum from 1 to N

• Write a subroutine to:

- Sum the positive numbers from 1 to **N**, where **N** is a value passed to the subroutine.
- The computed sum should be directly updated to a memory variable **Answer**, whose address is **0x100**.
- All parameters are to be passed via the **stack**.

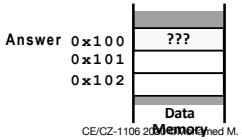
Solution:

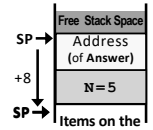
- Push two parameters on stack, the value of **N** and **address** of memory variable **Answer**.

Calling the Subroutine

```
      :                          ;find the sum of (1+2+3+4+5), where N=5
Parameters setup
      MOV   R1, #5               ;Set R0 with #5
      MOV   R0, #0x100           ;Set R1 with the address
      STMFD SP!, {r1,r0}         ;push R0&R1 to stack
      BL    Sum1N                ;call subroutine Sum1N
      ADD   SP,SP,#8             ;add 8 to pop the two parameters from stack
      :
```

Remove parameters from the Stack





Calling the Subroutine

```
      :                          ;find the sum of (1+2+3+4+5), where N=5
Parameters setup
      MOV   R1, #5               ;Set R0 with #5
      MOV   R0, #0x100           ;Set R1 with the address
      STMFD SP!, {R1,R0}         ;push R0&R1 to stack
      BL    Sum1N                ;call subroutine Sum1N
      ADD   SP,SP,#8             ;add 8 to pop the two parameters from stack
      :
```

Note:
1. Parameters set up before calling the subroutine are **removed** immediately after returning from subroutine.
2. Removal is done by returning the contents of the stack pointer to its **original value** before the parameters were pushed to the stack.

Sum from 1 to N Subroutine
Possible Solution

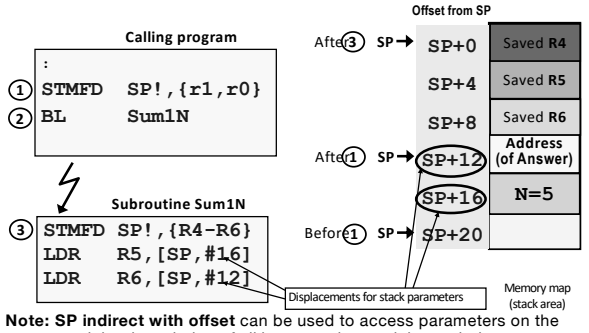
R6 Answer Addr
R5 N

```
Sum1N   STMFD  SP!,{R4,R5,R6}    ;save registers to stack
Retrieve stack parameters
        LDR    R5,[SP,#16]       ;Load N from stack to R5
        LDR    R6,[SP,#12]       ;Load Answer's From stack
        MOV    R4,#0             ;clear summation register R0 to 0
Loop    ADD    R4,R4,R5          ;add current value in R5 to R4
        SUBS   R5,R5,#1          ;decrement current value in R4 by 1
Z=0     BNE    Loop              ;jump back to Loop if R4 not yet zero
Z=1     STR    R4,[R6]           ;write sum to Answer
        LDMFD  SP!,{R4,R5,R6}    ;restored saved registers
        MOV PC,LR                ;return from subroutine
```

Note: The subroutine needs three registers. **R4** to compute the sum from 1 to N. **R6** to be an address pointer to the memory variable **Answer** where the results will written to. **R5** holds the value of N, which is decremented by 1 after each loop till it reaches 0.

Sum from 1 to N Subroutine

Accessing Stack Parameters



Note: SP indirect with offset can be used to access parameters on the stack but knowledge of all items on the stack is needed to compute the correct offset from the current **SP** position.

17

Transparent Subroutines

- A transparent subroutine will **not affect any CPU resources** used by the program calling it.
- To achieve this, all local registers used by the subroutine (R4-R11) must be **saved on the stack** on entry and **restored from stack** before returning.

```
SUB1 STMFD SP!,{R4-R7}    ; save R4 to R7 to stack
     :
     :                    ; registers R4 to R7 are
     :                    ; used in subroutine
     LDMFD SP!,{R4-R7}    ; restore R4 to R7 from stack
     MOV PC,LR            ; return to calling program
```


18

Passing by value and by reference

- Parameters are passed to subroutines in 2 ways:
  - **Pass by value** – the value of the data (or variable) is passed to the subroutine.
  - **Pass by reference** – the address of the variable is passed to the subroutine .

Calling program

```
MOV    R1,#5
MOV    R0,#0x100
STMFD  SP!,{R1,R0}
BL     Sum1N
```

- When is passing by reference used?
  - When the **parameter** passed is to be **modified** by the subroutine.
  - **Large quantity** of data (e.g. array) have to be passed between subroutine and calling program.

19

C Function Example

- **C** function to compute the mean value of **N** elements in an integer **Array**.

```
int Mean (int *Array, int N)
{
        int avg, i;
        int sum = 0;        } Local variables used only by the function
        for (i=0; i<N; i++)
          sum = sum + Array[i];
        avg = sum / N;
        return avg;         } Function's single output is normally returned via the R0 (or R12) register
}
```

Passing by reference — Passing by value

Note: Observe that **local variables** are required by the function to compute the result.

20

C Function Example

- **C** function to compute the mean value of **N** elements in an integer **Array**.
- Pass both parameters **by reference**.

```
int Mean (int *Array, int *N)
{
        int avg, i;
        int sum = 0;
        for (i=0; i<*N; i++)
          sum = sum + Array[i];
        avg = sum / *N;
        return avg;
}
```

Note: Observe that **local variables** are required by the function to compute the result.

21

Local Variables

- Subroutines often use local variables whose scope and **life span** exist only during the execution of the subroutine.

Review

- **Characteristics of a good software module.**
- Loose coupling – **data** within module is entirely **independent** of other modules (local variables).

22

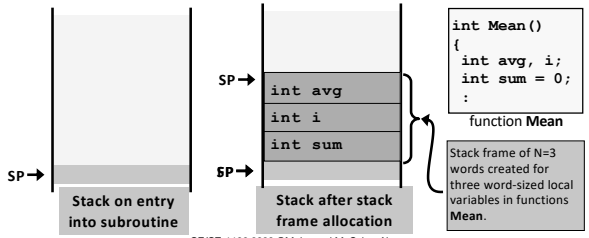
Local Variables

- Subroutines often use local variables whose scope and **life span** exist only during the execution of the subroutine.
  - Memory storage for these variables is **created on entry** into the subroutine and **released on exit** from subroutine.
  - The **system stack** is the ideal place to create memory space for temporary variables.
  - This temporary memory space is called the **stack frame**.

23

Stack Frame

- Stack frame of **N** words is created on the system stack immediately on entry into a subroutine.
- Memory space within the stack frame can be accessed using with a **frame pointer** (**FP**) or the stack pointer (**SP**).
- Frame pointer in ARM can be any register
  - General convention **FP** is **R11**

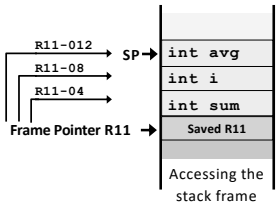


24

Accessing Stack Frame Variables

Using the Frame Pointer

- Consider the use of a frame pointer register **R11**.
  - Original contents of **R11** is saved on the stack before it is used as the frame pointer.
  - Frame pointer (**R11**) now points to the saved **R11** and a stack frame is created by adding frame size **4N** to **SP**.
  - **R11** is the frame reference and an appropriate negative displacement from **R11** can be used to access any stack frame variable.
  - When exiting the subroutine, the stack frame can be destroyed by adding **4N+4** to **SP** and copying the saved **R11** value on the stack back into **R11**.



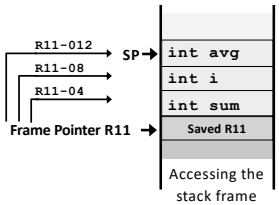
Accessing Stack Frame Variables

Using the Frame Pointer

- When exiting the subroutine, the stack frame can be destroyed by adding **4N+4** to **SP** and copying the saved **R11** value on the stack back into **R11**.

For N=3

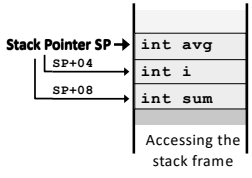
```
ADD SP, SP, #16
LDR R11,[R11]
```



Accessing Stack Frame Variables

Using the Stack Pointer

- A more efficient approach is to use the stack pointer (**SP**) itself.
  - A stack frame is created by adding frame size **4N** to **SP**.
  - **SP** is used as a reference to access all local variables.
  - Appropriate **positive** displacements from **SP** is used to access any of the stack frame variables.
  - **Pro** - This method is more efficient because there is no need to setup a frame pointer.
  - **Con** – More restrictive as system stack cannot be used within subroutine without changing the reference **SP**.



Summary

- Using the **stack** is the most favored means of passing parameters.
  - A combination of methods can be used to implement **Functions**, e.g. parameters passed in via stack and a single result value passed out via a register (e.g. **R0**).
  - Stack-based parameter passing supports **recursion**.
- Parameters is passed by **value** or **reference**.
  - Passing by reference allows the subroutine to directly access memory variables within the calling program .
- A **transparent** subroutine requires registers **used within** the routine to be saved on the **stack** on entry and restore before returning.
- Local variables within a subroutine are usually maintained on the system stack using a **stack frame**.